D10S Cup - Exposición del sistema y demostración en vivo

Guion del equipo | Taller de Construcción de Software | 2026 Duración sugerida: 12–15 minutos de exposición + 5 minutos de preguntas. Ajustar el reparto a las tareas realmente realizadas por cada integrante.

1. Objetivo de la presentación

Presentar el problema, objetivos y alcance; mostrar arquitectura y persistencia; recorrer funciones en funcionamiento; explicar seguridad, pruebas, presupuesto y límites; cerrar con próximos pasos. Diferenciar en todo momento lo que está implementado de lo planificado.

2. Roles y orden sugerido

1. Santiago Huechepan	2–3 min	Problema y objetivo; requisitos relevantes; arquitectura HTML/CSS/JS + Supabase/PostgreSQL/Auth; modelo, RLS y roles. Señalar que la propuesta inicial consideraba MySQL y la implementación usa Supabase/PostgreSQL.	“Con la base y permisos explicados, vemos la experiencia de consulta y competencia.”
2. Ignacio Marteniuk	2–3 min	Portada y navegación; torneos, búsqueda/modalidad y acción administrativa; límites de gestión de equipos/inscripciones; explicar cómo el rol restringe operaciones.	“Ahora mostramos el fixture y cómo se presentan los datos deportivos.”
3. Joaquín Namuncura	2–3 min	Partidos, carrusel/calendario; resultados, posiciones y fases eliminatorias; origen de datos; estados vacíos y limitación de carga en vivo.	“Cerramos el recorrido funcional con tienda y compra de demostración.”

4. Enzo Patiño	3–4 min	Tienda, variantes y carrito; sesión y compras propias; pedido, stock y ticket QR; aclarar pago simulado; diseño responsive, Vercel y próximos pasos.	“Después de la demo, resumimos pruebas, presupuesto y evolución.”

El reparto es una propuesta de presentación basada en módulos y no certifica el aporte individual. El equipo debe cambiar nombres/temas si su distribución real fue otra.

3. Guion de apertura

“D10S Cup es una plataforma web para organizar y consultar un torneo de fútbol y ofrecer una tienda asociada. El problema que buscamos atender es tener la información de competencias, fixture, resultados y productos en un solo lugar. Presentaremos las funciones conectadas y también los límites de esta versión para distinguir la demo académica de un servicio listo para operar pagos reales.”

4. Demo en vivo: recorrido de 5 minutos

Preparación previa

- Usar la URL de Preview o producción aprobada y comprobarla antes de exponer.
- Verificar red, Supabase, contenido y cuenta demo; tener preparada una segunda pestaña con portada.
- Confirmar que hay torneos, partidos, posiciones y productos visibles.
- No realizar checkout si el entorno no está identificado como demo o si no se autorizaron escrituras. Una compra altera stock y registros.
- No proyectar contraseñas, correo personal, claves API, SQL con credenciales ni paneles con información privada.
- Tener capturas locales como respaldo por caída de Internet; informar si se pasa al respaldo, sin presentarlo como vivo.

Pasos a narrar

1. Inicio: explicar objetivo, navegación y acceso a secciones (30 s).
2. Torneos: mostrar búsqueda/filtros y contenido conectado (45 s). Si se muestra alta, usar usuario admin y un torneo de prueba acordado; después eliminarlo según procedimiento de demo.
3. Partidos y resultados: mover carrusel/calendario; comparar una fila de posiciones con un resultado existente; abrir eliminatorias (60 s).
4. Cuenta: mostrar la sesión de demo y la pantalla de login/registro sin revelar credenciales; comentar recuperación (30 s). Si no hay correo de prueba listo, describir sin iniciar el flujo.
5. Tienda: buscar producto, mostrar variante/stock y carrito. Solo si existe autorización explícita y dataset de demo, completar un pedido y enseñar el comprobante/ticket QR e historial. Aclarar “pago simulado, sin transacción monetaria” (90 s).
6. Cierre: volver a portada, destacar que los datos provienen de Supabase y resumir siguientes pasos (30 s).

5. Puntos técnicos requeridos

Arquitectura

- Frontend estático con HTML semántico, CSS y módulos JavaScript.
- Supabase ofrece PostgreSQL, Auth y Data API; no hay backend HTTP propio.
- Vercel publica el sitio estático. Las variables públicas de Supabase se configuran como variables VITE_* al compilar.

Persistencia y seguridad

- Tablas relacionales para perfiles, equipos, competencias, productos, pedidos y tickets.

• RLS limita lecturas/escrituras por rol y propiedad. Las RPC de compra validan usuario, stock y precio en la base.
• La clave publicable se usa en navegador; jamás debe utilizarse service_role o sb_secret en el cliente.
• La confirmación simulada no comprueba un pago externo; para operar comercialmente hace falta proveedor y webhook confiable.

Pruebas y calidad

• Presentar el plan de pruebas: navegación, Auth, permisos, consistencia de posiciones, carrito, aislamiento de compras, contacto y manejo de errores.
• Informar solo resultados efectivamente ejecutados con fecha/evidencia. La existencia de un plan no significa que se ejecutó.
• Mostrar los casos pendientes y el proceso de registro de incidencias.

Presupuesto

• 340 horas estimadas, ARS 4.800/hora y total ARS 1.632.000 bajo los supuestos heredados.
• Es una estimación académica de referencia, no cotización actual. Pago real, infraestructura variable y mantenimiento requieren estimación aparte.
• El alcance completo presupuestado incluye módulos aún pendientes; no presentarlo como funcionalidad entregada.

6. Limitaciones y evolución

1. Integrar pasarela de pago real y confirmar con webhook firmado.
2. Validar tickets QR desde una interfaz de acceso.
3. Completar gestión de equipos, inscripciones, planteles y cupos.
4. Añadir administración de resultados en vivo y reportes.
5. Automatizar expiración de reservas y reforzar conciliación de pedidos.
6. Ejecutar QA integral en un entorno de staging y validar Auth/redirects del dominio final.

7. Preguntas probables y respuestas breves

?Por qué Supabase si el documento hablaba de MySQL? La implementación evolucionó a PostgreSQL administrado con Supabase, que integra Auth y políticas RLS; se debe actualizar la documentación para reflejar esa decisión.
?El pago es real? No. El sistema demuestra el flujo de pedido y ticket mediante confirmación simulada; no cobra dinero.
?Qué evita que alguien se haga administrador desde el navegador? El rol no se concede desde el cliente y las operaciones se vuelven a validar en PostgreSQL con RLS.
?El QR permite entrar al estadio? No en esta versión. Identifica un ticket; aún falta lector y validación de acceso.
?Qué falta para producción comercial? Pago real con webhook, política operativa de tickets/stock, pruebas completas, dominio y configuración de Auth, observabilidad y soporte.
?Cómo calculan las posiciones? La vista toma partidos jugados, agrega goles y resultados, y aplica 3 puntos por victoria y 1 por empate.

8. Cierre

“La versión actual demuestra una experiencia integrada de consulta y comercio de prueba, con datos persistentes y control de acceso en base. Los siguientes hitos para una operación real son completar la validación funcional, integrar pagos reales de forma segura y cerrar los módulos de administración pendientes.”